EAST WEST UNIVERSITY

CSE103: Structured Programming

Personal Expense Tracker

A simple C program to record, track, and total everyday expenses

Presented by: Mohammad Abdullah Bin Khan

Section: 19 · Semester: Summer 2026

What We'll Cover

01 **Problem & Objective**

Why this project, and what it sets out to do

02 **Key Features**

What the program can do

03 **Program Design & Flow**

Structure, array, control flow, and flowchart

04 **Validation & Error Handling**

How the program stays reliable

05 **Tools & Technologies**

What was used to build it

06 **Team Contribution**

Who did what

07 **Sample Run**

The program in action

08 **Learnings, Future Work & References**

What we took away, and what's next

Problem & Objective

THE PROBLEM

Students spend money every day but rarely track it. Expenses stay scattered across memory, paper notes, or nowhere at all — making it hard to know where the money actually went.

THE OBJECTIVE

Build a C console program that applies structured-programming concepts — arrays, structures, functions, loops, and file handling — to let a user add, view, search, update, and total their expenses, with data saved between runs.

Key Features

1

Add Expense

Log a new expense under a category with an amount

2

Display All

View every recorded expense in a clean table

3

Search by Category

Filter expenses to see spending in one area

4

Update Expense

Correct a category or amount by ID

5

Show Total

See the total amount spent so far

6

Save & Load

Data persists in expenses.txt between runs

Program Structure

One structure. One array. Seven main feature functions, each doing exactly one job.

```
struct Expense {
    int id;
    char category[20];
    float amount;
};

struct Expense list[MAX];
int count = 0;
```

WHY IT MATTERS

Grouping id, category, and amount into one struct — and keeping every record in a single array — is what lets the same simple pattern (loop through the array) power Display, Search, Update, and Total.

FUNCTIONS

`addExpense()` — create a new record

`displayExpense()` — show all records

`searchCategory()` — filter by category

`updateExpense()` — edit by ID

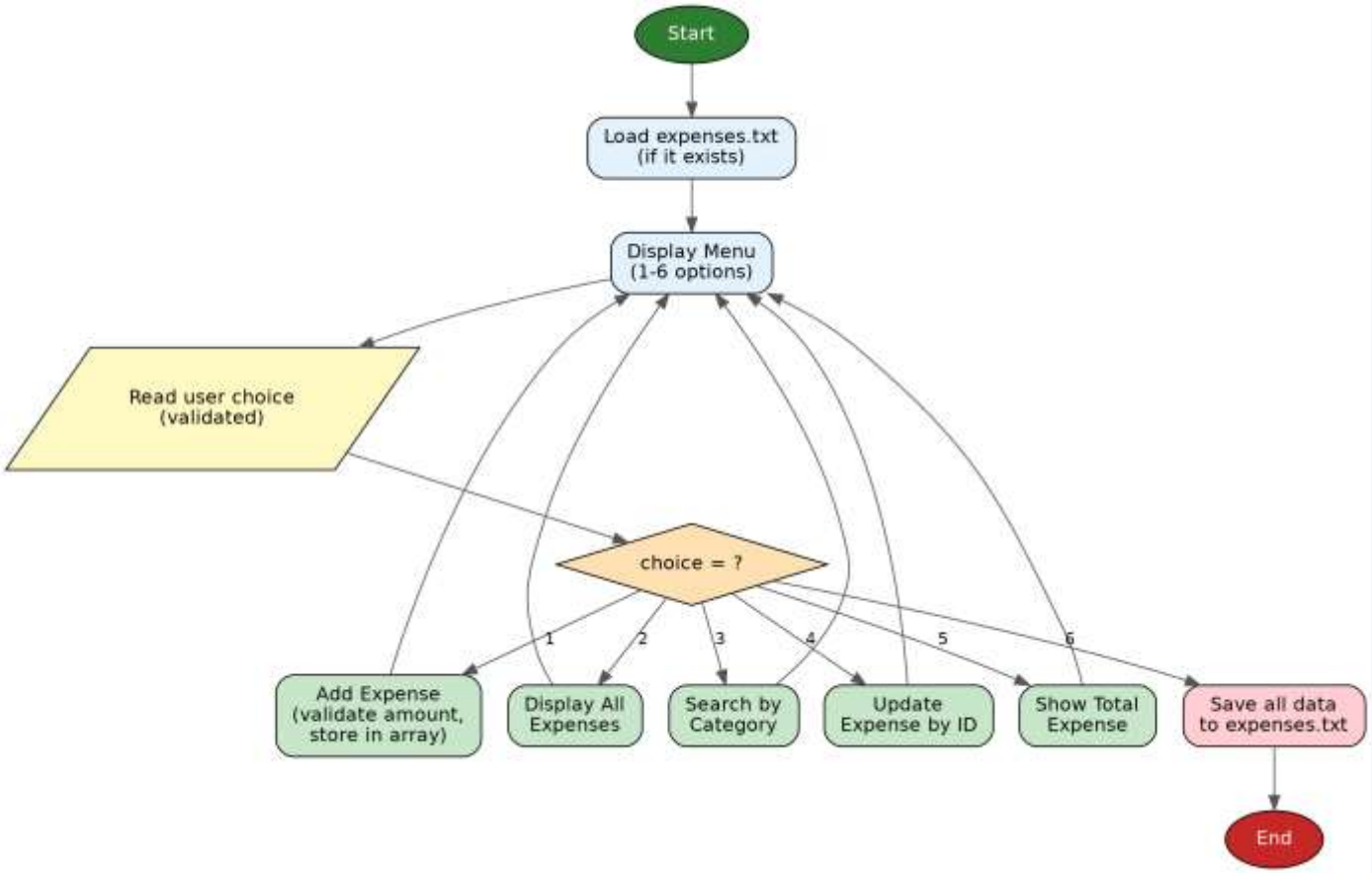
`totalExpense()` — sum all amounts

`saveFile() / loadFile()` — persistence

`readValidInt/Amount()` — input checks

Program Flow

Load saved data → show menu → run the chosen feature → loop back, until Save & Exit.



Input Validation & File Error Handling

OK

Input Validation

- Menu choice must be a real number (1-6) — text input is rejected and re-asked
- Amount must be a valid number, and cannot be negative
- Leftover bad input is cleared from the buffer so the next prompt isn't corrupted
- Category input is bounds-checked to prevent buffer overflow

FP

File Error Handling

- Every fopen() call is checked against NULL before use
- Saving: if the file can't be opened, the user sees a clear error instead of a crash
- Loading: a missing file just means "first run" — the program starts with an empty list
- Only fully-read records (id + category + amount) are added — a corrupted line stops the load

Tools & Technologies Used

1

C Compiler

GCC (GNU Compiler Collection) for compiling and running the program

2

IDE / Text Editor

Code::Blocks and Visual Studio Code for writing and debugging

3

Operating System

Developed and tested on Windows, with Linux compatibility in mind

4

Standard Libraries

stdio.h for input/output, string.h for string handling (strcmp)

Team Contribution & Role Assignment

Each member's focus area and share of the work.

Student ID	Student Name	Role	Contribution
2026-2-60-014	Mohammad Abdullah Bin Khan	Core Logic Development	40%
2026-2-60-034	Protik Paul	Input/Output Handling	20%
2026-2-50-038	Merajul Islam Provat	Testing & Debugging	20%
2026-2-50-013	Md. Jubayer Hossain	Documentation & Presentation	20%

Sample Run

```
===== EXPENSE TRACKER =====
1. Add Expense
2. Display All Expenses
3. Search by Category
4. Update Expense
5. Show Total Expense
6. Save & Exit
Enter choice: 2

ID    Category      Amount
1     Food          150.00
2     Transport     60.00
3     Food          90.00
```

What We Learned

- **Full-picture thinking**
  How structures, arrays, functions, and file I/O fit together in one real program
- **Validate early**
  Why validating input at the point of entry beats cleaning up bad data later
- **Defensive file I/O**
  How checking fopen()'s return value prevents silent crashes on file errors

FUTURE IMPROVEMENTS

Delete expenses · Monthly / category-wise summaries · Sort by amount or date

References & Resources

- **Textbook**

  Programming in ANSI C — E. Balagurusamy

- **Online Tutorials & Docs**

  W3Schools C Tutorial · GeeksforGeeks C Programming · Official C Standard Documentation

- **Project Repository**

  Full source code on GitHub — https://github.com/abkrohan/Personal-Expense-Tracker/

- **Class Lectures**

  CSE103: Structured Programming — lecture notes

Thank You